

计算机图形学

课程作业 3：物理渲染器

[方林升-102300311]

一、概述

本次作业主要围绕一个基于路径追踪算法的物理渲染器实现展开，整体流程包括光线生成与场景求交、空间加速结构、直接光照以及全局光照等模块。在总体框架上，我首先理解并搭建 `pathtracer` 的渲染流程：由相机发射光线、与场景求交、根据材质与光源计算辐射度，并通过多线程对图像划分为若干图块并行渲染，最终在 `sampleBuffer` 中累积得到高质量的图像。

在基础部分，我实现了相机射线生成和像素采样：根据给定的水平/垂直视角，将规范化的图像坐标映射到相机坐标系中的虚拟感光平面上，生成归一化的相机射线并变换到世界坐标系；随后在每个像素内利用随机采样生成多条相机光线，通过全局辐射度估计函数累积得到像素颜色。为完成场景求交，我分别实现了光线与三角形、球体以及包围盒的求交算法，并在此基础上填充交点信息（交点参数、法线、材质等），为后续的着色与加速结构遍历提供数据支撑。

为了提升复杂场景下的渲染效率，我构建了基于层次包围盒的 **BVH** 加速结构：递归地对场景图元按空间分布进行划分，为每个节点计算包围盒，并在图元数量小于阈值时生成叶子节点。渲染时，首先用射线与包围盒进行快速裁剪测试，仅对与射线相交的节点内图元做精确求交，从而将求交复杂度从线性降为对数级。

在光照计算方面，我实现了 **Lambert** 漫反射 **BSDF** 以及零反射照明与直接光照估计。零反射部分直接返回自发光表面的辐射度；在直接照明上，一方面实现了基于均匀半球采样的估计方法，另一方面实现了对面积光源/点光源进行重要

性采样的直接光照估计，通过对光源采样并结合可见性测试，有效降低噪声、提高收敛速度。

最后，在全局光照部分，我基于路径追踪实现了多次反射的辐射度累积：利用 `BSDF::sample_f` 对入射方向进行余弦加权采样，从交点继续发射新射线并递归求交与着色，实现最多 N 次反弹的间接光照；同时结合 Russian Roulette 作为随机终止条件，在保持估计无偏的前提下降低高深度路径的计算开销。综合零反射、一次反射与多次反射的贡献后，渲染器能够生成具有软阴影、间接漫反射以及颜色渗透等全局光照效果的照片级真实感图像。

二、光线生成与场景求交

光线投射（或路径追踪）的基本流程是：渲染器先在每个像素内生成若干子像素样本，把样本位置从像素坐标映射到归一化图像坐标，再根据相机的水平/垂直视场将其投到相机空间、距离小孔 1 个单位的感光平面上，由此得到从相机位置出发的射线方向。接着把这些射线送入场景，与几何图元（如三角形、球体）做求交，并始终保留最近且落在 $[\min_t, \max_t]$ 范围内的那个交点。若命中，记录交点位置、法线与材质（BSDF），根据直接光照与间接光照的估计求出该样本的辐射度；对同一像素聚合所有样本的结果取平均，得到最终像素颜色。

射线与三角形的相交通常用 Möller-Trumbore 算法。设三角形顶点为 a, b, c ，边向量 $e_1 = b - a$ 、 $e_2 = c - a$ ，射线为 $o + t d$ 。交点若存在，可写成 $o + t d = a + u e_1 + v e_2$ ，其中 $u, v \geq 0$ 且 $u + v \leq 1$ 。算法通过向量叉积与点积（等价于 Cramer 法则）直接解出 (t, u, v) ：先计算 $p = d \times e_2$ 与行列式 $\Delta = e_1 \cdot p$ ，若 $|\Delta|$ 极小则认为平行或退化；再算 $u = (o - a) \cdot p / \Delta$ ，若不在 $[0, 1]$ 则无交；接着令 $q = (o - a) \times e_1$ ，得到 $v = d \cdot q / \Delta$ ，若 $v < 0$ 或 $u + v > 1$ 也无交；最后 $t = e_2 \cdot q / \Delta$ ，只要 t 落在射线有效区间就判为命中。

```

1 Ray Camera::generate_ray(double x, double y) const {
2
3     double tanHalfH = tan(radians(hFov) * 0.5);
4     double tanHalfV = tan(radians(vFov) * 0.5);
5
6     Vector3D p_cam(
7         (2.0 * x - 1.0) * tanHalfH,
8         (2.0 * y - 1.0) * tanHalfV,
9         -1.0
10    );
11
12    Vector3D dir_world = (c2w * p_cam).unit();
13
14    Ray r(pos, dir_world);
15
16    return r;
17 }

```

图 2-1：任务 1.1 的代码实现

```

1 void PathTracer::raytrace_pixel(size_t x, size_t y) {
2
3     int num_samples = ns_aa;
4
5     Vector3D L(0.0, 0.0, 0.0);
6
7     for (int i = 0; i < num_samples; ++i) {
8         Vector2D s = gridSampler->get_sample();
9
10        double u = (x + s.x) / sampleBuffer.w;
11        double v = (y + s.y) / sampleBuffer.h;
12
13        Ray r = camera->generate_ray(u, v);
14        L += est_radiance_global_illumination(r);
15    }
16
17    L /= (double)num_samples;
18
19    sampleBuffer.update_pixel(L, x, y);
20    sampleCountBuffer[x + y * sampleBuffer.w] = num_samples;
21 }
22 }

```

图 2-2：任务 1.2 的代码实现

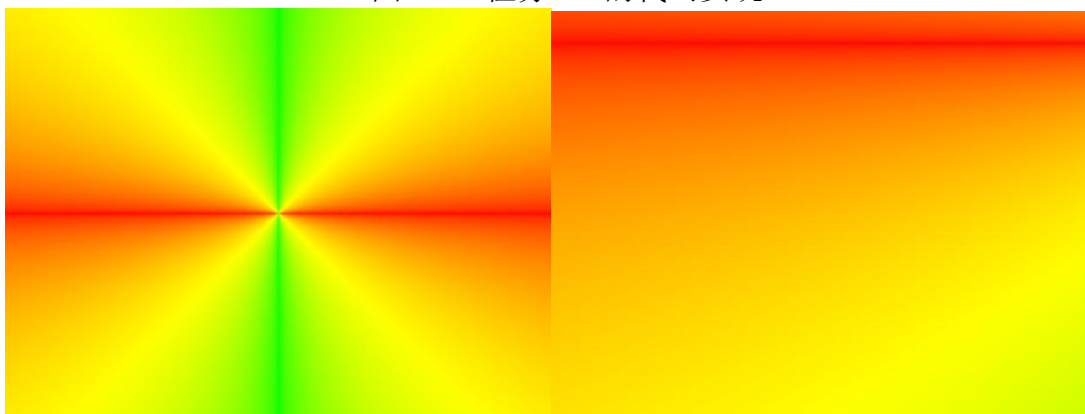


图 2-3：任务 1.2 的实现效果

```

1 bool Triangle::hasIntersection(const Ray &r) const {
2     const Vector3D e1 = p2 - p1;
3     const Vector3D e2 = p3 - p1;
4
5     const Vector3D pvec = cross(r.d, e2);
6     const double det = dot(e1, pvec);
7     const double EPS = 1e-9;
8     if (fabs(det) < EPS) return false;
9
10    const double invDet = 1.0 / det;
11    const Vector3D tvec = r.o - p1;
12    const double u = dot(tvec, pvec) * invDet;
13    if (u < 0.0 || u > 1.0) return false;
14
15    const Vector3D qvec = cross(tvec, e1);
16    const double v = dot(r.d, qvec) * invDet;
17    if (v < 0.0 || u + v > 1.0) return false;
18
19    const double t = dot(e2, qvec) * invDet;
20    if (t < r.min_t || t > r.max_t) return false;
21
22    return true;
23 }
24
25 }
26
27 bool Triangle::intersect(const Ray &r, Intersection *isect) const {
28     const Vector3D e1 = p2 - p1;
29     const Vector3D e2 = p3 - p1;
30
31     const Vector3D pvec = cross(r.d, e2);
32     const double det = dot(e1, pvec);
33     const double EPS = 1e-9;
34     if (fabs(det) < EPS) return false;
35
36     const double invDet = 1.0 / det;
37     const Vector3D tvec = r.o - p1;
38     const double u = dot(tvec, pvec) * invDet;
39     if (u < 0.0 || u > 1.0) return false;
40
41     const Vector3D qvec = cross(tvec, e1);
42     const double v = dot(r.d, qvec) * invDet;
43     if (v < 0.0 || u + v > 1.0) return false;
44
45     const double t = dot(e2, qvec) * invDet;
46     if (t < r.min_t || t > r.max_t) return false;
47
48     r.max_t = t;
49     isect->t = t;
50
51     const double w = 1.0 - u - v;
52     isect->n = (w * n1 + u * n2 + v * n3).unit();
53
54     isect->primitive = this;
55     isect->bsdf = get_bsdf();
56
57     return true;
58 }
59
60 }

```

图 2-4: 任务 1.3 的代码实现

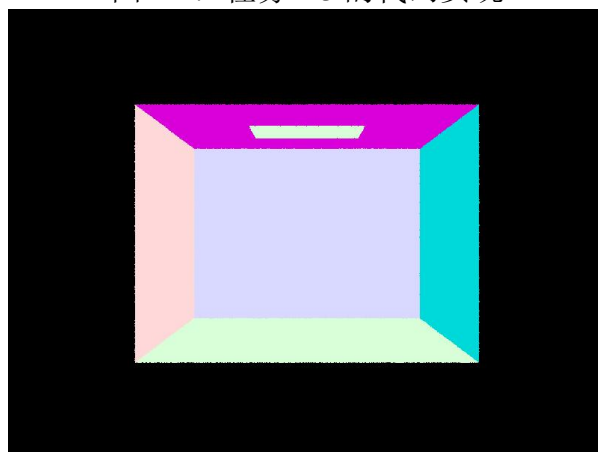


图 2-5: 任务 1.3 的实现效果

```

1
2 bool Sphere::test(const Ray &r, double &t1, double &t2) const {
3
4     Vector3D oc = r.o - o;
5     double a = dot(r.d, r.d);
6     double b = 2.0 * dot(r.d, oc);
7     double c = dot(oc, oc) - r2;
8
9     double disc = b * b - 4.0 * a * c;
10    if (disc < 0.0) return false;
11
12    double s = sqrt(disc);
13    double inv2a = 0.5 / a;
14    t1 = (-b - s) * inv2a;
15    t2 = (-b + s) * inv2a;
16    if (t1 > t2) std::swap(t1, t2);
17
18    return true;
19
20 }
21
22 bool Sphere::has_intersection(const Ray &r) const {
23
24     double t1, t2;
25     if (!test(r, t1, t2)) return false;
26
27     double t = t1;
28     if (t < r.min_t) t = t2;
29
30     if (t < r.min_t || t > r.max_t) return false;
31
32     return true;
33 }
34
35 bool Sphere::intersect(const Ray &r, Intersection *i) const {
36
37     double t1, t2;
38     if (!test(r, t1, t2)) return false;
39
40     double t = t1;
41     if (t < r.min_t) t = t2;
42     if (t < r.min_t || t > r.max_t) return false;
43
44     r.max_t = t;
45     i->t = t;
46     Vector3D p = r.o + t * r.d;
47     i->n = (p - o).unit();
48     i->primitive = this;
49     i->bsdf = get_bsdf();
50
51     return true;
52 }

```

图 2-6：任务 1.4 的代码实现

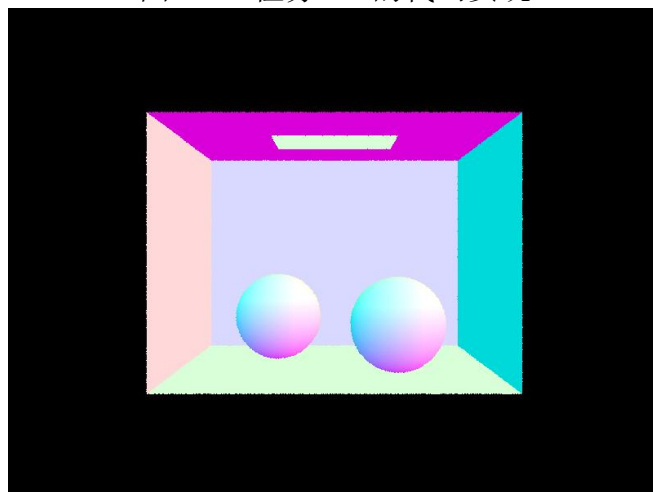


图 2-7：任务 1.4 的实现效果

三、空间加速数据结构 (以 BVH 为例)

首先，采用自顶向下的递归建树：对当前区间 $[start, end)$ 的图元先计算其包围盒并创建一个 BVHNode。若图元数量 $n \leq \text{max_leaf_size}$ ，则把该节点标记为叶子（记录迭代器 $start/end$ ），递归在此终止；否则将节点继续二分。

二分时，我使用基于质心的最长轴划分。具体做法是：先计算所有图元质心的包围盒（centroid bbox），选取其最长轴作为划分轴；将划分平面放在该 centroid bbox 的中心位置，然后按“质心在平面左/右”把图元用 `std::partition` 原地分到两侧。这样能让两个子节点尽量覆盖各自紧凑的空间，同时避免频繁复制数据。

为避免退化（例如所有质心恰好在同一侧或轴向范围极小），如果一次中点划分导致全部图元落到同一边，我会退化为等量划分：沿同一轴用 `std::nth_element` 取中位数，把区间按数量 1:1 拆分，确保每次递归至少减少一半元素，避免无限递归或极度不平衡的树。

```
1 BVHNode *BVHAccel::construct_bvh(std::vector<Primitive *>::iterator start,
2 std::vector<Primitive *>::iterator end,
3 size_t max_leaf_size) {
4
5     BBox bbox;
6     for (auto p = start; p != end; ++p) {
7         bbox.expand((*p)->get_bbox());
8     }
9
10    BVHNode *node = new BVHNode(bbox);
11
12    size_t n_prims = end - start;
13    if (n_prims <= max_leaf_size) {
14        node->start = start;
15        node->end = end;
16        return node;
17    }
18
19    BBox centroid_bbox;
20    for (auto p = start; p != end; ++p) {
21        centroid_bbox.expand((*p)->get_bbox().centroid());
22    }
23
24    Vector3D e = centroid_bbox.extent();
25    int axis = 0;
26    if (e.y > e.x && e.y > e.z) axis = 1;
27    else if (e.z > e.x && e.z > e.y) axis = 2;
28
29    double split_pos = centroid_bbox.centroid()[axis];
30
31    auto mid = std::partition(start, end,
32                             [axis, split_pos](const Primitive *prim) {
33                                 return prim->get_bbox().centroid()[axis] < split_pos;
34                             });
35
36    if (mid == start || mid == end) {
37        auto mid_equally = start + n_prims / 2;
38        std::nth_element(start, mid_equally, end,
39                        [axis](const Primitive *a, const Primitive *b) {
40                            return a->get_bbox().centroid()[axis] <
41                                b->get_bbox().centroid()[axis];
42                        });
43        mid = mid_equally;
44    }
45
46    node->l = construct_bvh(start, mid, max_leaf_size);
47    node->r = construct_bvh(mid, end, max_leaf_size);
48    return node;
49 }
```

图 3-1：任务 2.1 的代码实现

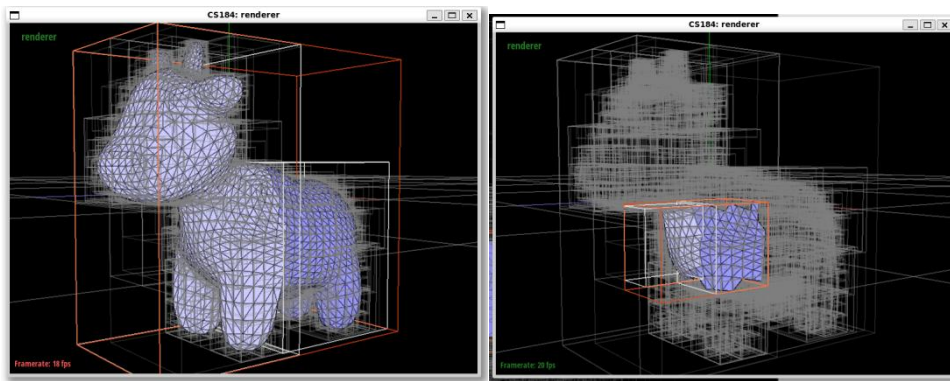


图 3-2: 任务 2.1 的实现效果

```

1 bool BBox::intersect(const Ray& r, double& t0, double& t1) const {
2
3     double t_enter = t0;
4     double t_exit = t1;
5
6     for (int axis = 0; axis < 3; ++axis) {
7         double o = r.o[axis];
8         double d = r.d[axis];
9
10        if (d == 0.0) {
11            if (o < min[axis] || o > max[axis]) return false;
12            continue;
13        }
14
15        double invD = 1.0 / d;
16        double tmin_axis = (min[axis] - o) * invD;
17        double tmax_axis = (max[axis] - o) * invD;
18        if (tmin_axis > tmax_axis) std::swap(tmin_axis, tmax_axis);
19
20        t_enter = std::max(t_enter, tmin_axis);
21        t_exit = std::min(t_exit, tmax_axis);
22
23        if (t_enter > t_exit) return false;
24    }
25
26    t_enter = std::max(t_enter, r.min_t);
27    t_exit = std::min(t_exit, r.max_t);
28
29    if (t_enter > t_exit) return false;
30
31    t0 = t_enter;
32    t1 = t_exit;
33    return true;
34

```

图 3-3: 任务 2.2 的代码实现

```

1  bool BVHAccel::has_intersection(const Ray &ray, BVHNode *node) const {
2      double t0 = ray.min_t, t1 = ray.max_t;
3      if (!node->bb.intersect(ray, t0, t1)) return false;
4
5      if (node->isLeaf()) {
6          Ray r_clip = ray;
7          r_clip.min_t = t0; r_clip.max_t = t1;
8          for (auto p = node->start; p != node->end; ++p) {
9              total_isects++;
10             if ((*p)->has_intersection(r_clip)) return true;
11         }
12         return false;
13     }
14
15     double lt0 = ray.min_t, lt1 = ray.max_t;
16     double rt0 = ray.min_t, rt1 = ray.max_t;
17     bool hitL = node->l->bb.intersect(ray, lt0, lt1);
18     bool hitR = node->r->bb.intersect(ray, rt0, rt1);
19
20     if (hitL && hitR) {
21         if (lt0 <= rt0) {
22             Ray rl = ray; rl.min_t = lt0; rl.max_t = lt1;
23             if (has_intersection(rl, node->l)) return true;
24             Ray rr = ray; rr.min_t = rt0; rr.max_t = rt1;
25             return has_intersection(rr, node->r);
26         } else {
27             Ray rr = ray; rr.min_t = rt0; rr.max_t = rt1;
28             if (has_intersection(rr, node->r)) return true;
29             Ray rl = ray; rl.min_t = lt0; rl.max_t = lt1;
30             return has_intersection(rl, node->l);
31         }
32     } else if (hitL) {
33         Ray rl = ray; rl.min_t = lt0; rl.max_t = lt1;
34         return has_intersection(rl, node->l);
35     } else if (hitR) {
36         Ray rr = ray; rr.min_t = rt0; rr.max_t = rt1;
37         return has_intersection(rr, node->r);
38     }
39     return false;
40
41
42 }
43
44 bool BVHAccel::intersect(const Ray &ray, Intersection *i, BVHNode *node) const {
45     double t0 = ray.min_t, t1 = ray.max_t;
46     if (!node->bb.intersect(ray, t0, t1))
47         return false;
48     if (node->isLeaf()) {
49         bool hit = false;
50         for (auto p = node->start; p != node->end; ++p) {
51             total_isects++;
52             if ((*p)->intersect(ray, i)) {
53                 hit = true;
54             }
55         }
56         return hit;
57     }
58
59     bool hit_left = intersect(ray, i, node->l);
60     bool hit_right = intersect(ray, i, node->r);
61
62     return hit_left || hit_right;
63 }

```

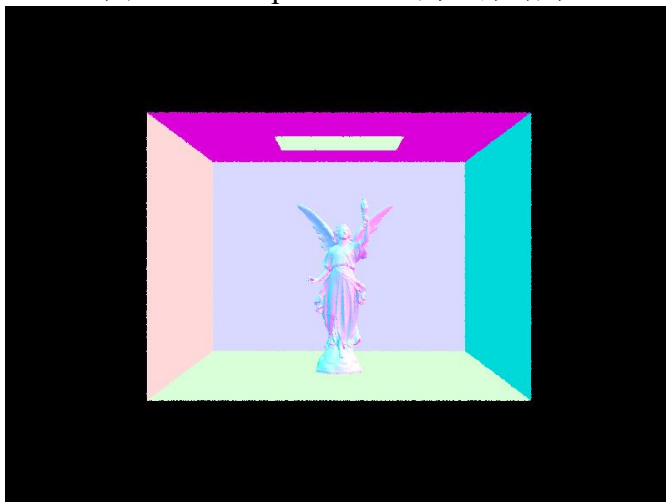
图 3-4: 任务 2.3 的代码实现



图 3-5: cow.dae 的渲染结果



图 3-6: maxplanck.dae 的渲染结果



、图 3-7: CBlucy.dae 的渲染结果

对比使用和未使用 BVH 的运行时间，如表 1 所示：

表 1：运行时间对比

选择的网络	未使用 BVH 运行时间	使用 BVH 运行时间
cow.dae	27.4295s	0.0588s
maxplanck.dae	247.7484s	0.1661s
CBlucy.dae	644.5879s	0.3416s

四、直接光照

直接光照（direct illumination）只计算相机光线首次命中点从可见光源直接到达该点、再经由该点反射到相机的光。它对应渲染方程中的一项：

$$L_o(p, \omega_o) = \int_{\Omega^+} f_r(p, \omega_i \rightarrow \omega_o) L_i(p, \omega_i) \cos \theta_i d\omega_i$$

其中 f_r 是 BSDF， L_i 是来自光源的入射辐射度， $\cos \theta_i$ 是入射方向与法线的夹角余弦，仅在可见半球 Ω^+ 上累积。与“间接光”不同，它不包含经场景多次反弹后的光。

数值上通常用蒙特卡罗估计这项积分。做法是对半球或对光源采样入射方向 ω_i ，发一条“阴影射线”检测该方向是否被遮挡（可见性项）。若没有遮挡，就把该样本的贡献

$$\frac{f_r L_i \cos \theta}{p(\omega_i)}$$

累加起来并对样本数取平均，其中 $p(\omega_i)$ 是采样该方向的概率密度。

采样策略影响噪声与效率。半球均匀采样实现简单，但很多方向并不指向高亮区域，方差大；重要性采样直接对每个光源（或其表面）采样方向，样本更集中于真正有贡献的方向，方差更低。对点光/方向光（delta 光）通常只需一次样本；对面积光按其几何与辐射分布采样多个点，并将距离衰减与法线投影（ $\cos \theta$ ）一并计入。

因此，直接光照的核心流程就是：建立命中点的局部坐标系 → 采样入射方向（选择合适的 PDF）→ 发阴影射线做可见性测试 → 在局部用 BSDF 评估并按 $\cos \theta / p$ 加权累积。重复多次取平均即可得到稳定的直接光估计。

```
1 Vector3D DiffuseBSDF::sample_f(const Vector3D wo, Vector3D *wi, double *pdf) {  
2  
3     *wi = sampler.get_sample(pdf);  
4     return f(wo, *wi);  
5  
6 }
```

```
1 Vector3D DiffuseBSDF::f(const Vector3D wo, const Vector3D wi) {  
2  
3     return reflectance / PI;  
4  
5 }
```

图 4-1：任务 3.1 的代码实现

```
1 Vector3D PathTracer::zero_bounce_radiance(const Ray &r,  
2                                           const Intersection &isect) {  
3     return isect.bsdf->get_emission();  
4 }
```

```
1 Vector3D PathTracer::est_radiance_global_illumination(const Ray &r) {  
2     Intersection isect;  
3     Vector3D L_out;  
4  
5     if (!bvh->intersect(r, &isect))  
6         return envLight ? envLight->sample_dir(r) : L_out;  
7  
8     return zero_bounce_radiance(r, isect);  
9 }
```

图 4-2：任务 3.2 的代码实现

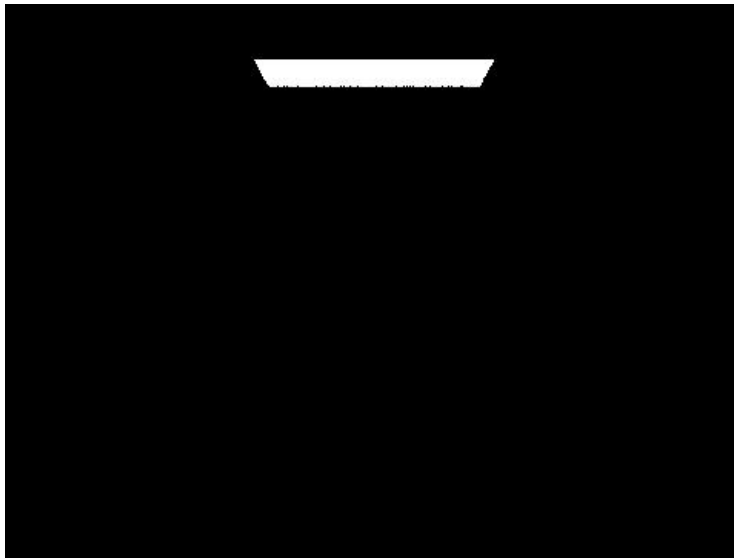


图 4-3: 任务 3.2 的实现效果

```
1 Vector3D PathTracer::one_bounce_radiance(const Ray &r,  
2                                           const Intersection &isect) {  
3     return direct_hemisphere_sample  
4         ? estimate_direct_lighting_hemisphere(r, isect)  
5         : estimate_direct_lighting_importance(r, isect);  
6 }
```

```
1 Vector3D PathTracer::est_radiance_global_illumination(const Ray &r) {  
2     Intersection isect;  
3     Vector3D L_out;  
4  
5     if (!bvh->intersect(r, &isect))  
6         return envLight ? envLight->sample_dir(r) : L_out;  
7  
8     L_out = zero_bounce_radiance(r, isect) + one_bounce_radiance(r, isect);  
9  
10    return L_out;  
11 }
```

图 4-4: 任务 3.3 的代码实现 1

```

1 Vector3D
2 PathTracer::estimate_direct_lighting_hemisphere(const Ray &r,
3                                                  const Intersection &isect) {
4     Matrix3x3 o2w; make_coord_space(o2w, isect.n);
5     Matrix3x3 w2o = o2w.T();
6
7     const Vector3D hit_p = r.o + r.d * isect.t;
8     const Vector3D w_out = w2o * (-r.d);
9
10    int num_samples = std::max(1, (int)scene->lights.size() * (int)ns_area_light);
11    Vector3D L_out;
12    const double pdf_hemi = 1.0 / (2.0 * PI);
13
14    for (int i = 0; i < num_samples; ++i) {
15        Vector3D wi_local = hemisphereSampler->get_sample();
16        if (wi_local.z <= 0.0) continue;
17
18        Vector3D wi_world = o2w * wi_local;
19
20        Ray shadow(hit_p + EPS_F * isect.n, wi_world);
21        shadow.min_t = EPS_F;
22        shadow.max_t = INF_D;
23
24        Intersection isl;
25        Vector3D li(0);
26        if (bvh->intersect(shadow, &isl)) {
27            li = isl.bsdf ? isl.bsdf->get_emission() : Vector3D();
28        } else {
29            li = envLight ? envLight->sample_dir(shadow) : Vector3D();
30        }
31        if (li == Vector3D()) continue;
32
33        L_out += isect.bsdf->f(w_out, wi_local) * li * cos_theta(wi_local) / pdf_hemi;
34    }
35    return L_out / (double)num_samples;
36 }

```

图 4-5：任务 3.3 的代码实现 2

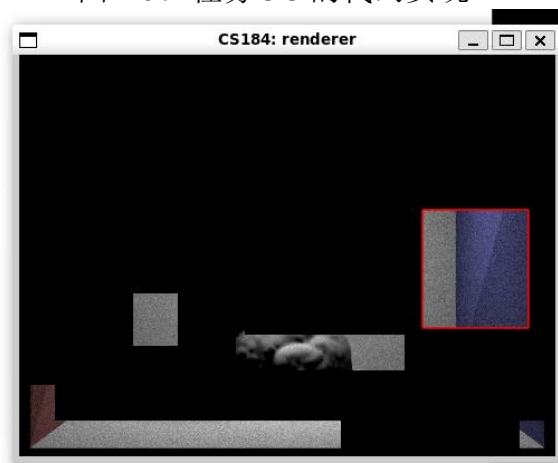


图 4-6：使用单元格渲染功能预览效果

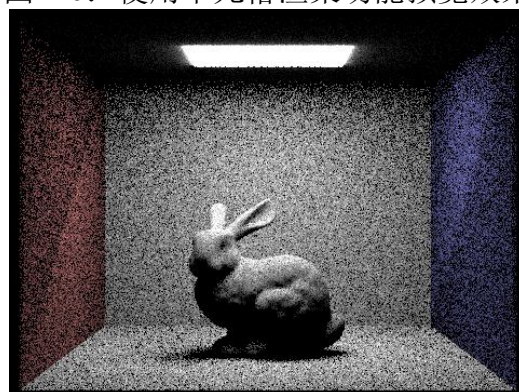


图 4-7：低采样率渲染任务 3.3 的效果

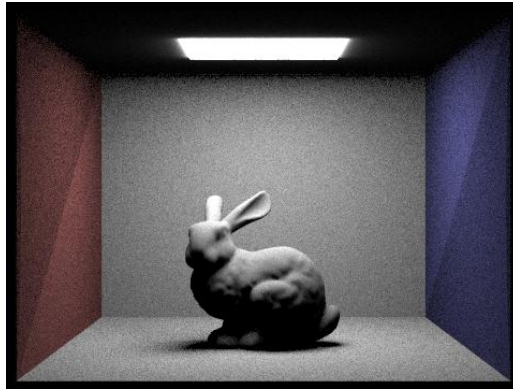


图 4-8：高采样率渲染任务 3.3 的效果

```

1  Vector3D
2  PathTracer::estimate_direct_lighting_importance(const Ray &r,
3                                                    const Intersection &isect) {
4
5      Matrix3x3 o2w;
6      make_coord_space(o2w, isect.n);
7      Matrix3x3 w2o = o2w.T();
8
9      const Vector3D hit_p = r.o + r.d * isect.t;
10     const Vector3D w_out = w2o * (-r.d);
11     Vector3D L_out;
12
13     for (auto light = scene->lights.begin(); light != scene->lights.end(); light++) {
14         Vector3D wi_world;
15         double distToLight;
16         double pdf;
17         double cosTheta;
18         int numSamples = 0;
19         Vector3D sampleLight;
20         Vector3D accumLight;
21
22         for (int i = 0; i < ns_area_light; i++) {
23             sampleLight = (*light)->sample_L(hit_p, &wi_world, &distToLight, &pdf);
24             cosTheta = dot(isect.n, wi_world);
25
26             Ray shadowRay = Ray(hit_p, wi_world, 1);
27             shadowRay.min_t = EPS_F;
28             shadowRay.max_t = distToLight - EPS_F;
29             Intersection newIntersect;
30             if (cosTheta > 0 && !(bvh->intersect(shadowRay, &newIntersect))) {
31                 accumLight += sampleLight * isect.bsdf->f(w_out, wi_world) * cosTheta / pdf;
32             }
33             numSamples++;
34
35             if ((*light)->is_delta_light()) {
36                 break;
37             }
38         }
39         L_out += accumLight / numSamples;
40     }
41 }
42
43 return L_out;
44
45 }

```

图 4-9：任务 3.4 的代码实现



图 4-10：任务 3.4 的低采样率渲染效果

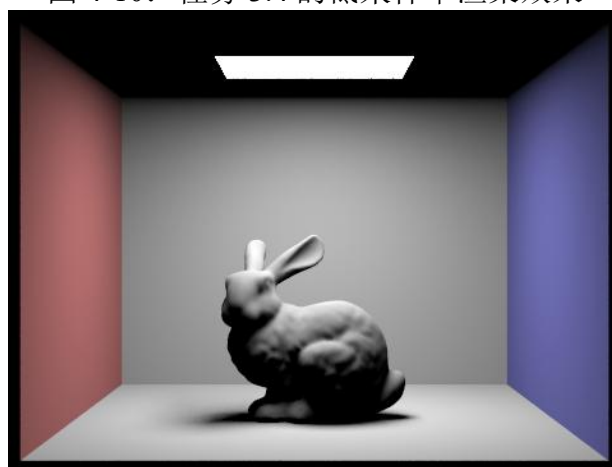


图 4-11：任务 3.4 的高采样率渲染效果

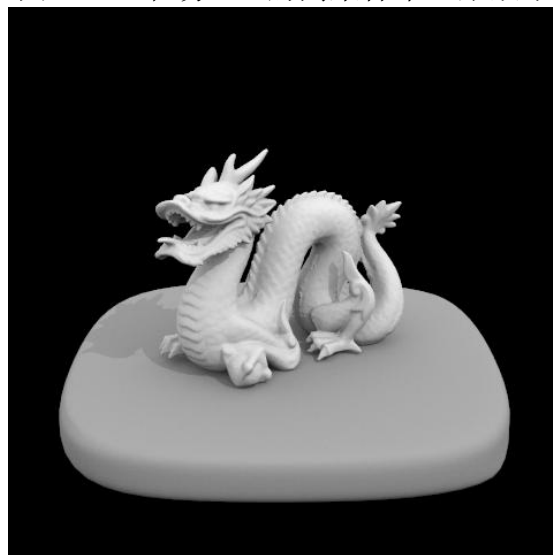


图 4-12：dragon.dae 的渲染效果

图 4-8 是典型的半球均匀采样效果，图 4-11 则对应光源做重要性采样的结果。两张图在整体光照和明暗关系上是一致的——说明两种方法在理论上都是无偏估计，同样能收敛到正确解——但在收敛速度和噪声水平上差别巨大。

在半球均匀采样中，每个可见点都是在整个上半球随机选一个方向去发射光线，命中光源视为贡献，没有命中就是零贡献。这样做有两个问题：

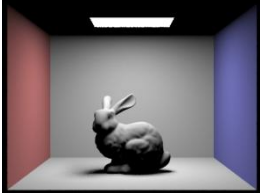
- 1. 光源只占整个半球中很小的一块立体角，大量样本都“打空”，白白浪费；
- 2. 亮度集中在光源附近，而采样方向是均匀的，导致大部分样本贡献很小、极少数样本贡献很大，方差很高，表现出来就是图 4-8 那种大面积的颗粒噪点，尤其是在灰墙和天花板附近。

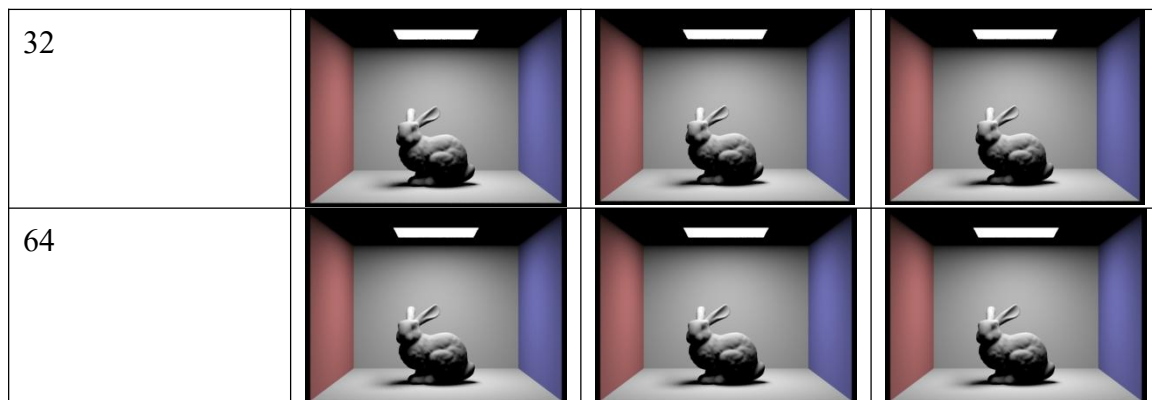
对光源做重要性采样则是反过来思考：既然真正决定亮度的是光源，就直接在“指向光源”的方向附近采样。也就是根据光源在半球上所占的立体角及其亮度分布来设计采样概率密度函数，让采样方向更集中地指向光源。这样一来：

- 1. 绝大多数样本都能命中光源或靠近光源方向，具有较大的有效贡献；
- 2. 各个样本的贡献值相对接近，方差大幅降低。所以在图 4-11 中，即使在相同的每像素样本数下，墙面、地面和兔子的阴影都显得非常平滑，几乎看不到噪点。

从渲染效率的角度看，两种方法的期望值相同，但半球均匀采样需要数量级更多的样本才能达到左图那样的干净效果；而重要性采样在相同预算下就能获得更好的信噪比。这就是蒙特卡洛积分中“把采样分布做得越接近真实贡献分布，收敛越快”的直观体现。在实际路径追踪器中，通常会对光源、BRDF（如余弦加权、Phong lobe 等）同时做重要性采样，并配合多重重要性采样，以在各种场景下都获得类似图 4-11 这样的高质量结果。

表 2：不同参数下的渲染图像质量对比

-s\l	16	32	64
16			



五、全局光照

全局光照的基本思想，是“光线会在场景中多次反射和传播”。传统的局部光照模型只考虑从光源直接打到物体表面，再进入眼睛（或相机）的那一条路径，例如 Phong 或 Blinn-Phong，只算直射光和简单高光，忽略了光在场景中到处弹射的过程。而真实世界中，一束光线通常会在不同物体之间多次反射、折射与散射，最后才进入我们的眼睛，这些多次反弹产生的间接光照就叫做全局光照。

在数学上，全局光照可以用“渲染方程”描述：某点向某个方向的出射光等于该点的自发光（比如发光物体本身）加上来自所有方向的入射光乘以该点材质的反射/折射特性后，在该方向上的贡献。因为每个点的入射光又来自其他点的出射光，这就形成了一个互相耦合的积分方程，直接精确求解非常困难。

路径追踪（Path Tracing）是解决渲染方程的一种常用数值方法。它从相机发射很多条“随机光线”，模拟光线在场景中的传播：每次光线击中一个表面，根据该表面的 BSDF（反射分布函数）随机采样一个下一步方向，继续追踪，直到光线击中光源或被终止。把这些随机路径的贡献在像素上平均，就得到对真实光照的蒙特卡洛估计。这样不仅能模拟直接光照，还能自然表现出颜色溢出（color bleeding）、柔和阴影、间接高光等真实效果。

为了控制计算量，路径追踪会设定最大反弹次数，并使用 Russian Roulette（俄罗斯轮盘）随机终止光线，把被留下的路径贡献适当放大，以保证整体估计

保持无偏。通过足够多的样本以及合适的重要性采样策略，全局光照算法可以在满足物理能量守恒的前提下，逐渐逼近真实世界中的光照效果。

```
1 Vector3D DiffuseBSDF::sample_f(const Vector3D wo, Vector3D *wi, double *pdf) {
2
3     *wi = sampler.get_sample(pdf);
4     if (wo.z < 0.0) {
5         wi->z *= -1.0;
6     }
7     return f(wo, *wi);
8
9 }
```

图 5-1：任务 4.1 的代码实现

```
1 Vector3D PathTracer::est_radiance_global_illumination(const Ray &r) {
2     Intersection isect;
3     Vector3D L_out;
4
5     if (!bvh->intersect(r, &isect)) {
6         return envLight ? envLight->sample_dir(r) : L_out;
7     }
8
9     if (isAccumBounces) {
10         L_out = zero_bounce_radiance(r, isect) +
11                at_least_one_bounce_radiance(r, isect);
12     } else {
13         L_out = at_least_one_bounce_radiance(r, isect);
14     }
15
16     return L_out;
17 }
18
```

图 5-2：任务 4.2 的代码实现 1

```

1 Vector3D PathTracer::at_least_one_bounce_radiance(const Ray &r,
2                                                    const Intersection &isect) {
3     Matrix3x3 o2w;
4     make_coord_space(o2w, isect.n);
5     Matrix3x3 w2o = o2w.T();
6
7     Vector3D hit_p = r.o + r.d * isect.t;
8     Vector3D w_out = w2o * (-r.d);
9
10    Vector3D L_out(0.0);
11    if (isAccumBounces || (r.depth + 1 == (int)max_ray_depth)) {
12        L_out += one_bounce_radiance(r, isect);
13    }
14
15    if (r.depth + 1 >= (int)max_ray_depth) {
16        return L_out;
17    }
18
19    Vector3D wi;
20    double pdf = 0.0;
21    Vector3D f = isect.bsdf->sample_f(w_out, &wi, &pdf);
22
23    if (pdf <= 0.0 || wi.z == 0.0) {
24        return L_out;
25    }
26
27    double cos_theta = abs_cos_theta(wi);
28    if (cos_theta <= 0.0) {
29        return L_out;
30    }
31
32    Vector3D wi_world = o2w * wi;
33    Ray new_ray(hit_p + EPS_F * wi_world, wi_world);
34    new_ray.min_t = EPS_F;
35    new_ray.max_t = INF_D;
36    new_ray.depth = r.depth + 1;
37
38    Intersection next_isect;
39    Vector3D Li(0.0);
40
41    if (bvh->intersect(new_ray, &next_isect)) {
42        Vector3D L_emit = zero_bounce_radiance(new_ray, next_isect);
43        Vector3D L_multi = at_least_one_bounce_radiance(new_ray, next_isect);
44        Li = L_emit + L_multi;
45    } else if (envLight) {
46        Li = envLight->sample_dir(new_ray);
47    }
48
49    L_out += f * Li * (cos_theta / pdf);
50
51    return L_out;
52 }

```

图 5-3: 任务 4.2 的代码实现 2

```

1 Vector3D PathTracer::at_least_one_bounce_radiance(const Ray &r,
2                                                     const Intersection &isect) {
3     Matrix3x3 o2w;
4     make_coord_space(o2w, isect.n);
5     Matrix3x3 w2o = o2w.T();
6     Vector3D hit_p = r.o + r.d * isect.t;
7     Vector3D w_out = w2o * (-r.d);
8
9     Vector3D L_out(0.0);
10
11     if (isAccumBounces || (r.depth + 1 == (int)max_ray_depth)) {
12         L_out += one_bounce_radiance(r, isect);
13     }
14
15     if (r.depth + 1 >= (int)max_ray_depth) {
16         return L_out;
17     }
18
19     const double p_rr = 0.35;
20     bool use_rr = (r.depth > 0);
21     double survival_prob = 1.0;
22
23     if (use_rr) {
24         survival_prob = 1.0 - p_rr;
25         if (coin_flip(p_rr)) {
26             return L_out;
27         }
28     }
29
30     Vector3D wi;
31     double pdf = 0.0;
32     Vector3D f = isect.bsdf->sample_f(w_out, &wi, &pdf);
33
34     if (pdf <= 0.0 || f == Vector3D()) {
35         return L_out;
36     }
37
38     double cos_theta = abs_cos_theta(wi);
39     if (cos_theta <= 0.0) {
40         return L_out;
41     }
42
43     Vector3D wi_world = o2w * wi;
44     Ray new_ray(hit_p + EPS_F * wi_world, wi_world);
45     new_ray.min_t = EPS_F;
46     new_ray.max_t = INF_D;
47     new_ray.depth = r.depth + 1;
48
49     Intersection next_isect;
50     Vector3D Li(0.0);
51
52     if (bvh->intersect(new_ray, &next_isect)) {
53         Vector3D L_emit = zero_bounce_radiance(new_ray, next_isect);
54         Vector3D L_multi = at_least_one_bounce_radiance(new_ray, next_isect);
55         Li = L_emit + L_multi;
56     } else if (envLight) {
57         Li = envLight->sample_dir(new_ray);
58     }
59
60     double weight = use_rr ? (1.0 / survival_prob) : 1.0;
61     L_out += f * Li * (cos_theta * weight / pdf);
62
63     return L_out;
64 }

```

图 5-4: 任务 4.3 的代码实现

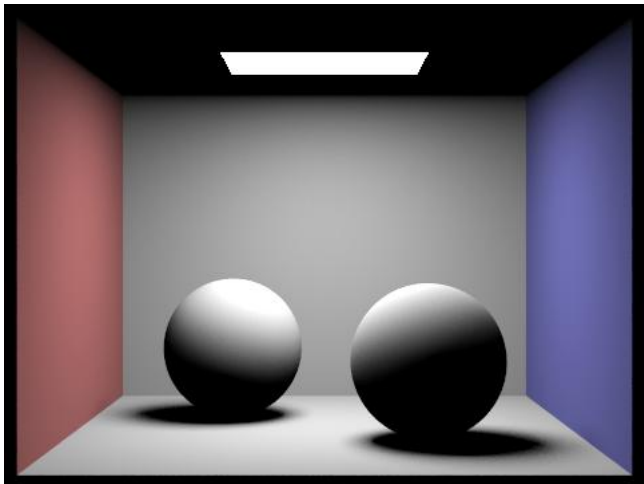


图 5-5: CBspheres.png 的全局光照渲染效果

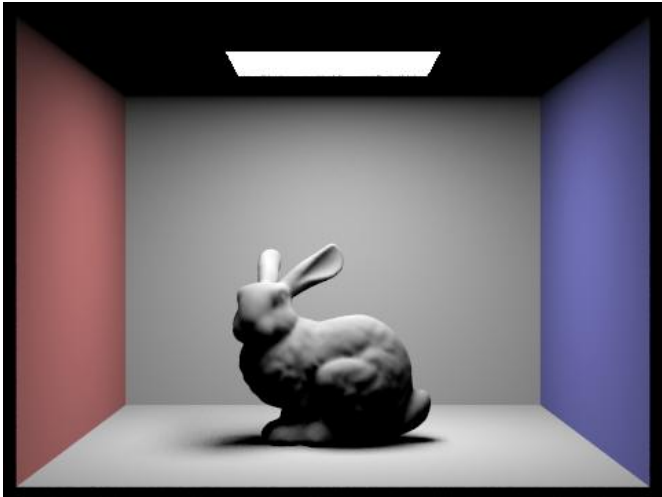
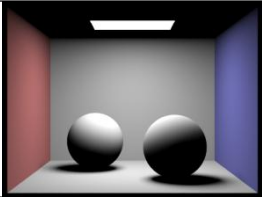
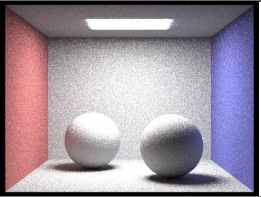
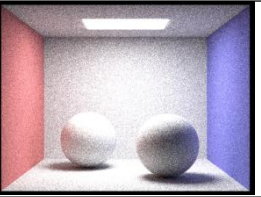


图 5-6: CBbunny.png 的全局光照渲染效果

表 3: 不同参数下全局光照的渲染效果

最大光路深度	1	2	3
渲染效果			

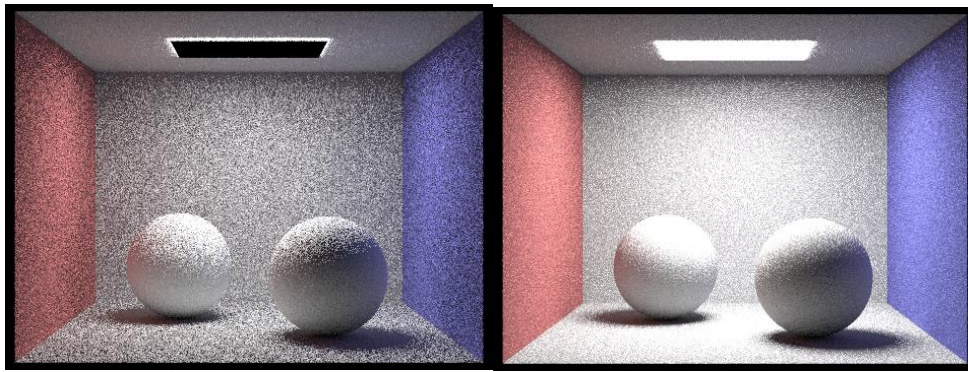


图 5-7：直接光照（左）与全局光照（右）的对比

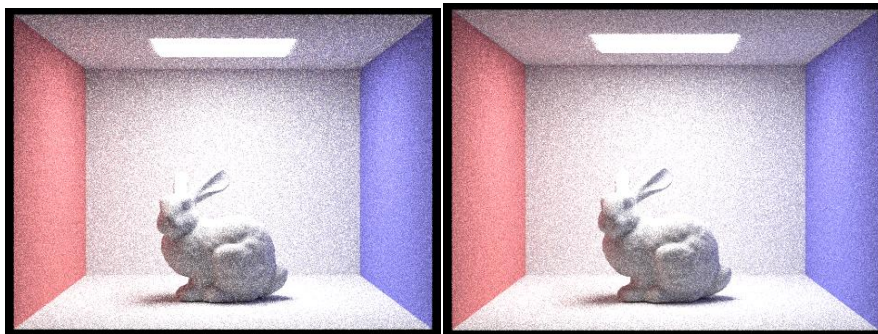


图 5-8：使用 Russian Roulette（左）与没使用（右）的对比

六、额外工作

6.1 其他材质的实现

我实现了镜面反射（Mirror）、微表面反射（Microfacet）、纯折射（Refraction）、玻璃（Glass），以及它们依赖的两个辅助函数 `BSDF::reflect` 和 `BSDF::refract`。这些 BSDF 都是在局部坐标系下工作的：法线固定是 $(0,0,1)$ ， $z>0$ 表示在表面上方的半球， $\cos_theta(v)$ 实际上就是 $v.z$ 。

首先是 `MirrorBSDF`。它代表理想镜面，所以理论上 BRDF 是一个“ δ 分布”，只在精确的反射方向有能量，其他方向都为 0。因此 `f()` 直接返回 0，而在 `sample_f()` 中，我们调用 `BSDF::reflect(wo, wi)` 得到唯一的镜面反射方向，把 pdf 设为 1（因为这是一个确定性的采样），然后返回值用 `reflectance / |cos $\theta_i$ |`，这里除以 `abs_cos_theta(*wi)` 是为了抵消路径追踪器里乘上的余弦项，保证能量在这种 delta BSDF 下是正确的。

然后是比较核心的 MicrofacetBSDF。它采用 Beckmann 法线分布函数来描述微表面的统计分布。在 $D(h)$ 中，我们先通过 $\cos_theta(h)$ 计算出半角向量的入射角，再由 $\tan^2 \theta = (1 - \cos^2 \theta) / \cos^2 \theta$ 得到 $\tan^2 \theta_h$ ，代入 Beckmann 的公式

$$D(h) = \frac{\exp(-\tan^2 \theta_h / \alpha^2)}{\pi \alpha^2 \cos^4 \theta_h}$$

这里 α 是粗糙度参数。 $G(w_o, w_i)$ 使用头文件里定义的 Λ 函数，按照 Smith 模型写成 $1 / (1 + \Lambda(w_o) + \Lambda(w_i))$ ，考虑遮蔽和自阴影。 $F(w_i)$ 实现的是导体界面的精确菲涅尔公式：对每个颜色通道，分别用 (η, κ) 计算 R_s 和 R_p ，再取平均 $(0.5 * (R_s + R_p))$ 。最终在 $f()$ 中，只要 w_o 、 w_i 都在上半球，就构造半角向量 $h = (w_o + w_i).unit()$ ，然后根据标准微表面公式

$$f_r = \frac{F D G}{4 \cos \theta_o \cos \theta_i}$$

返回结果。

Microfacet 的采样在 $sample_f()$ 中实现。我们从 $[0,1]^2$ 的随机数 $(u.x, u.y)$ 出发，对 Beckmann 分布进行重要性采样：利用

$$\tan^2 \theta = -\alpha^2 \ln(1 - u_1), \quad \phi = 2\pi u_2$$

求出 (θ, ϕ) ，再还原为半角向量 h 的三维坐标。然后，把出射方向 w_o 关于 h 反射得到 $w_i = 2(w_o \cdot h)h - w_o$ ，如果 w_i 跑到下半球就拒绝。概率密度先在半角空间得到 $pdf(h) = D(h) * \cos \theta_h$ ，再通过雅可比变换转成方向空间： $pdf(w_i) = pdf(h) / (4 * (w_o \cdot h))$ 。最后调用 $f(w_o, w_i)$ 返回 BRDF 值，实现了对 Beckmann NDF 的重要性采样。

RefractionBSDF 表示理想的纯折射。同样地， $f()$ 返回 0，因为是 delta 型 BSDF；真正的工作在 $sample_f()$ 里完成。它先根据 w_o 和折射率 ior 判断是从空气进介质还是相反，设定好 η_i 、 η_t 之后调用 $BSDF::refract(w_o, w_i, ior)$ 。一旦出现全反射（函数返回 false），我们就把 pdf 设为 0，认为没有折射路径；否则 $pdf = 1$ ，并用 $transmittance / (|\cos \theta_i| * \eta^2)$ 作为返回值，其中 $\eta = \eta_i / \eta_t$ ，

$1/\eta^2$ 是折射时的能量缩放（立体角压缩）， $/|\cos\theta_i|$ 依旧是为了抵消路径追踪器中的余弦项。

GlassBSDF 同时包含反射和折射，是一个“玻璃材质”。在 `sample_f()` 中，我们先尝试用 `BSDF::refract` 计算折射方向，如果失败则说明发生了全内反射，此时必然走镜面反射分支。折射成功时，我们用 Schlick 近似计算一个标量的 Fresnel 反射率：

$$R_0 = \left(\frac{\eta_i - \eta_t}{\eta_i + \eta_t} \right)^2, \quad R = R_0 + (1 - R_0)(1 - \cos\theta)^5$$

然后以 R 的概率选择镜面反射，以 $1-R$ 的概率选择折射。反射分支里调用 `reflect`，`pdf = R`，返回 $R * \text{reflectance} / |\cos\theta_r|$ ；折射分支里直接使用刚才算好的折射方向 `wt`，`pdf = 1 - R`，返回 $(1-R) * \text{transmittance} / (|\cos\theta_t| * \eta^2)$ 。这样的设计保证了 Fresnel 加权的能量守恒：取样概率和贡献强度匹配。

最后两个辅助函数 `BSDF::reflect` 和 `BSDF::refract` 封装了几何上的向量运算。`reflect` 在局部坐标下法线为 $(0,0,1)$ ，因此只需要把 x 、 y 分量取反： $(-w_o.x, -w_o.y, w_o.z)$ 。`refract` 则严格按照 Snell 定律：根据 $w_o.z$ 判断是进入还是射出介质，设置 η_i 和 η_t ，计算 $\sin^2\theta_i$ 和 $\sin^2\theta_t = (\eta_i/\eta_t)^2 \sin^2\theta_i$ ，若 $\sin^2\theta_t \geq 1$ 则判定全内反射；否则算出 $\cos\theta_t$ ，用切向分量缩放 η ， z 分量改为适当符号的 $\pm\cos\theta_t$ ，组合成折射方向 w_i 。这两个函数被 `Mirror`、`Refraction` 和 `Glass` 复用，既减少重复代码，也使整体逻辑更加清晰。

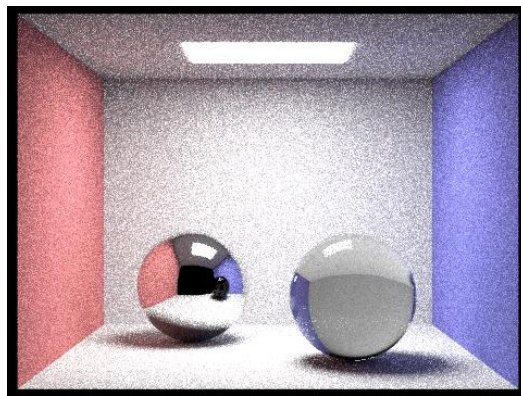


图 6-1：实现效果

6.2 “FZU”三个字母的渲染效果



图 6-2: FZU

七、实践与思考

在编程和调试过程中，我基本是按模块逐步推进、边实现边验证的节奏来完成整个渲染器的。最开始在实现相机射线生成和基础求交时，经常出现“全黑画面”或者物体轮廓明显错位的问题，后来通过在控制台打印关键变量、渲染简单测试场景（只有一个三角形或球体），一点点排查坐标变换和 t 取值区间，才确认射线方向、近远平面以及相交判定都正确。随着 BVH、直接光照和全局光照等模块逐步加入，我会在每个阶段先渲染课程提供的基准场景，对比是否能得到类似的几何形状和明暗关系，再继续往后叠加功能。

在调试 BVH 加速结构时，问题主要集中在“有的模型三角形消失”或者“某个角落被莫名其妙地切掉”。这类 Bug 往往不是崩溃，而是渲染结果肉眼看起来不对。我通过先关闭 BVH 与开启 BVH 的结果做对比，再配合打印 BVH 结构统计信息（节点数量、深度、叶子中图元数量），最终定位到是划分轴与质心范围处理不当导致一侧为空，从而出现退化树或漏检。修正为“质心中点划分 + 退化时

数量均分”后，渲染结果恢复正常，同时在复杂场景上运行时间从数百秒降到不到 1 秒，性能提升非常直观。

在光照与材质部分，调试的重点更多是“数值正确但效果很吵”。例如刚实现半球均匀采样的直接光照时，画面整体亮度正确，但墙面和阴影区域噪点很多；全局光照和玻璃材质加入后，高光附近还会出现严重的“亮斑（fireflies）”。通过对比不同采样策略的结果，我逐渐意识到重要性采样和合理的 pdf 归一是降噪的关键：对面积光单独采样、在 `BSDF::sample_f` 中按理论公式返回 $f * \cos / \text{pdf}$ ，可以大幅降低噪声；同时合理设置最大反弹次数、在较大深度启用 Russian Roulette，也能在不明显影响亮度的前提下减少无效路径，缩短渲染时间。

从整体实验效果来看，目前渲染器已经能较好地表现软阴影、颜色溢出以及玻璃/金属等不同材质的视觉差异，说明路径追踪整体框架和能量传输是基本正确的。但仍然存在一些问题：低采样率下噪声较重，高光和折射区域偶尔会出现孤立亮点；在极复杂场景中，CPU 端渲染时间依然偏长。针对这些问题，进一步的改进思路包括：在直接光照和 BSDF 采样之间引入多重重要性采样（MIS），进一步抑制噪声；对路径贡献做适度的能量截断，减少极端样本带来的 fireflies；在实现层面可以考虑更精细的 BVH 构建策略或并行化优化（如更合理地划分任务块），在保持图像质量的同时进一步压缩渲染时间。通过这些改进，预期可以在相同采样数下得到更干净的图像，或者在保持画面质量不变的前提下降低渲染成本。